

I underestimated AI capabilities (again)

Revisiting a prediction ten months early



AJEYA COTRA

MAR 05, 2026



87



25



9

On Jan 14th, I [made predictions](#) about AI progress in 2026. My forecasts for engineering capabilities already feel much too conservative.

In my view, [METR](#) (where I now work) has some of the hardest and highest-software engineering and ML engineering benchmarks out there, and the new framework for making benchmark performance intuitive: we measure a task's difficulty by the amount of time a human expert would take to complete it (the [“time horizon”](#)). ¹

When I made my forecasts last month, the model with the longest measured time horizon on METR's suite of software engineering tasks was Claude Opus 4. It succeeded around half the time at software tasks that would take a human software engineer about five hours. ² Time horizons on software tasks had been doubling less than twice a year from 2019 through 2025, which would have implied that the state-of-the-art 50% time horizon should be somewhat less than 20 hours by the end of 2025. But there was ambiguity about whether the more recent doubling time was the long-run trend, so I bumped that up to 24 hours for my median guess. ⁴ My 25th percentile was around 15 hours and my 80th percentile was around 40 hours.

Now, Opus 4.6 (released only 2.5 months after Opus 4.5) was estimated to have a time horizon of ~12 hours. ⁵ I don't take the specific number literally — there are fewer very-long tasks than medium and short tasks, and the long tasks more guesstimated (rather than measured) human completion times, so time hori

estimates for the latest models are a lot noisier than they were in 2025. And benchmark underlying the time horizon graph is nearly saturated, which causes confidence intervals to blow up: the 95% CI is 5.3 hours to 66 hours.⁶ It's really hard to discriminate between different capability levels at the current range.

But at the end of the day, that dataset had 19 software engineering tasks estimated to take humans longer than 8 hours, and Opus 4.6 was able to solve 14 of them in some of the time (and it reliably nailed four of them).⁸ And beyond just this suite, we've seen examples of AI agents doing certain very well-specified software tasks like writing a browser or C compiler, or porting a giant game, that would take humans many weeks or months to do on their own — not perfectly, but better than most people expected and better than a naive reading of the agents' measured time horizon would have suggested.

And this happened in February. It's no longer very plausible that after ten weeks of additional progress at the recent blistering pace,⁹ AI agents would struggle half the time at 24 hour tasks.

I wish them the best, but I think my colleagues on the capability evaluation METR might struggle to create new software tasks from a similar distribution of measuring AI agents' true time horizons through the end of the year. If we measure this, I'd guess that by the end of the year, AI agents will have a time horizon of over 100 hours on the sorts of software tasks in METR's suite (which are precisely specified — on certain extremely well-specified software tasks like the examples above, agents seem to *already* have a time horizon of more than a week of work).

And once you're talking about multiple full-time-equivalent *weeks* of work, the whole concept of "time horizon" starts to break down.

It's nearly impossible to subdivide a typical one hour task (e.g., debugging a test) into smaller pieces that multiple people can work on in parallel. It would

very well if you had to farm out writing this print statement or reading that message or tweaking this line of code to different people — the right action depends intimately on everything that came before it and the precise state of the world as a whole, you have to hold the whole context in your mind as you take each step, or they won't cohere in the right way.

It's somewhat easier to decompose an eight hour task (e.g., writing a simple game) into smaller components, but those components are constantly bleeding into each other in ways that make clean handoffs hard. When you're implementing game logic, you realize it needs to know something about how the graphics are rendered. When you're handling user input, you find yourself tweaking the graphics. The fastest way to do it is probably one person knocking it out in a day, making a hundred small decisions fluidly as they go.

But it's actually pretty feasible to break down a *month*-long task into smaller pieces. In fact, you may start *benefiting* from some explicit decomposition — it might be easier to write a design doc laying out how the pieces fit together, or break the work into tickets so you don't lose track of what's done and what's left. And while it might take one person working alone a month to complete, the fastest way to get it done is to have different people work on different pieces like the checkout flow or the inventory management panel in parallel.

And of course, tasks that take multiple full-time-equivalent *years* of work need to be broken down into smaller milestones, and parallelized across multiple teammates. Human work appears to get more and more decomposable the longer and longer it gets.

In other words, very few tasks feel *intrinsically* like year-long tasks, the way that one bash command feels like an intrinsically one-second task, or debugging a simple bug feels intrinsically like a one-hour task. Maybe a mathematician banging their head against a hard conjecture for a year before finally making a breakthrough is a “real” year-long task? But most many-person-month software projects in

world sort of feel like they might be a bunch of few-week tasks in a trenchcoat way that a hundred-question elementary school math test is really 100 thirty tasks in a trenchcoat.

If an extreme version of that is true, then once AI agents can consistently do hour tasks, they should be able to make continuous progress on projects of scale. Maybe manager AIs can spend their work week figuring out how to fit current project goal to line-worker AIs, line-workers can execute on their in piece, and all the AIs can maintain good enough records that no individual needs to build up holistic, long-term state on the whole project.

I think this *probably* won't fully work. Even projects with lots of formalized tracking still benefit a *lot* from everyone involved intuitively appreciating the picture in a way that isn't fully captured in Jira tickets and Asana tasks. Doing a 6 month project so cleanly and precisely that it can be executed by a team with no such holistic context might itself be, say, a 2 month task.

But it might work surprisingly well for a surprisingly large class of software. AI agents are a lot cheaper and a *lot* more patient than humans, so it could lead to get them to do far, *far* more task-tracking, documentation, and other project management than human teams ever do. People have already started aggressively experimenting with scaffolding for orchestrating agent teams. It's not clear how well it will go over the next several months.

This is why my colleague Tom **proposed** that the calendar time it takes a large team of humans to do a task might be a better proxy for "intrinsic difficulty" than the time it takes one human working alone. So far, the "team time" and "solo time" have been very similar in the METR task suite, since it has ranged from 1 second tasks to 20 hour tasks. But we're entering the regime where these numbers could rapidly diverge. If Tom's conjecture is true, the "solo time" metric should start going up exponentially about now...which makes it very hard to bound software engineering capabilities by the end of the year.

In my predictions [last month](#), my probability that AI R&D would be *fully* automated by the end of the year — AIs taking care of all the research ideation and implementation with no humans necessary¹⁰ — was 10%. After I published that piece, I heard from others in the AI forecasting space (including those I generally think of as more optimistic about AI timelines than I am) that it seemed a bit high. But now ten percent feels much more in the right ballpark again.

Fully automating AI R&D still seems like a tall order. Even fully automating engineering seems like it requires an aggressive read of the evidence, and AI is not just software engineering — it seems like automating it would require a significant amount of progress on “research judgment” and “creativity” and other epistemic skills that AI systems still appear to be worse at than human researchers. It is a lot more likely in the coming three or five years than this year.

But for the first time, I don’t see any solid trend we can extrapolate to say it will happen soon.¹¹ AI R&D really could be automated *this year*.

¹ It takes a bit of taste and judgment to decide what a task’s “time horizon” is, and it is possible to game the metric to make it meaningless. Consider the “task” of taking a math test consisting of 10,000 easy elementary school word problems — this is composed of 10,000 thirty-second tasks, rather than one 83-hour task. Or to take an example from my colleague Tom Cunningham, consider the task “Count how many times a horse is mentioned in *Anna Karenina*.” This might take a single human 10 hours, but it’s parallelizable: a team of 300 people could each take one page and do the task in 10 minutes. In the METR task suite, “human time to complete” works as a good proxy for “intrinsic difficulty” because the tasks are constructed to be hard to easily decompose and parallelize. Within each task, every piece depends on every other, and you benefit from keeping the whole context in mind. More on that later in the post.

- 2 Actually, at the time, METR was measuring models on its original time horizon 1.0), and the precise central estimate for Opus 4.5 was ~4h48min on that distribution. Since then, METR has released an updated suite with more tasks (TH 1.1) which shifts models' scores slightly; the measured time horizon for Opus 4.5 on TH 1.1 is ~5h20min.
- 3 The 2019-2025 doubling time calculated in the [original paper](#) was 212 days, or 0.58 years. That means that if the time horizon at the beginning of January was 5 hours, the time horizon at the end of the year should be $5 * \exp(\ln(2) * 1/0.58) \approx 16.4$ hours. When doing mental math, I was approximating the 7 month doubling time as two doublings: $5 * 2 * 2 \approx 20$ hours. The rule of thumb was a bit more aggressive than the strict exponential extrapolation, but in fact, I just realized while writing this post that the mental extrapolation was too *conservative* in a different way — I should have dated the time horizon to Nov 24 (Opus 4.5's release date), not the beginning of January, meaning I should have added an extra month to all these extrapolations.
- 4 I didn't actually do this math at the time, but the [original paper](#) calculated a doubling time of 118 days or 0.32 years since 2024, and if you assume that doubling time was correct, the time horizon by EOY 2026 should have been $5 * \exp(\ln(2) * 1/0.32) \approx 43$ hours. I took this faster doubling time pretty seriously at this time, this suggests my model was especially my 80th percentile should have been higher to begin with. I would probably have been better served by the heuristic of using just the previous year (rather than the previous seven years) to forecast the next year.
- 5 Originally, METR estimated Opus 4.6 to have a 50% time horizon of ~14.5 hours in early 2026. We [corrected a bug in our modeling](#) on March 3, 2026 and this reduced its time horizon estimate to ~12 hours. Note that this measurement was done on Time Horizon 1.1, the new task suite released on Jan 29, 2026.

- 6 This is something I appreciate about the time horizon construct. For standard benchmarks, confidence intervals around a point get *narrower* as the benchmark approaches a model gets 50% accuracy, there's more room for error in either direction than if a model gets 90% accuracy. But since the time horizon metric could get infinitely long, the confidence intervals can be constructed so uncertainty gets *wider* rather than narrower as the hardest tasks are saturated. Epistemically, it's appropriate for your uncertainty about AI capabilities to blow up when you no longer have tasks that the models can't complete.
- 7 In the original time horizon [paper](#), METR measured human completion times for the tasks in the dataset by actually having humans do them (148 out of 169). In this suite, only 5 of the 19 tasks longer than 8 hours have measured human baselines; for the others, they are estimated.
- 8 Agents are usually run on the same task 6 separate times. The same agent doesn't approach the same task the same way each time — it sometimes gets lucky or unlucky. For four of the 19 hard tasks, Claude Opus 4.6 solved it successfully in all six runs; for the other five, it solved it successfully in at least one of the six runs.
- 9 If you look just at the year 2025, agent time horizons doubled every [~3.5 months](#), which is much faster than months as in the long-run trend or even every ~4 months as in the 2024-2025 trend. I didn't factor this into my forecasts; it wasn't salient to me compared to the rule of thumb I absorbed of "two doublings a year." If I had used this to extrapolate from Opus 4.6, I would have suggested time horizons by the end of the year should be $5 * \exp(\ln(2) * 12) \approx 54$ hours by the end of the year.
- 10 Specifically, my operationalization was that firing all human members of technical research (research leads, engineers, everyone) would only slow down progress by 25%. No doubt in the past, of course, firing every single human would cause progress to completely halt.
- 11 A concrete example of this is the kind of argument that my colleague Nikola made in Nov 2025, when the state-of-the-art time horizon was 2 hours; this kind of argument would have been much shakier made today (less than four months later!).

Subscribe to Planned Obsolescence

By Ajeya Cotra · Launched a year ago

Thinking ahead to a future where AI decides everything

Type your email...	Subscribe
--------------------	-----------

By subscribing, you agree Substack's [Terms of Use](#), and acknowledge its [Information Collection Notice](#) and [Privacy Policy](#).



87 Likes · 9 Restacks

← Previous

Discussion about this post

Comments Restacks



Write a comment...



Will Kiely 5d

Even if 50% Success is going super-exponential now, wouldn't you expect 80% Success to go exponential before full automation of AI R&D happens?



LIKE (7)



REPLY

12 replies



Jacob Asmuth 5d

Agreed with everything written in the comments to you so far. Lots of useful feedback to iterate on!

1. We already measure 80%, so simply stop reporting 50% as it is clearly becoming therefore less useful.
2. 80% seems to lag 50% by nearly a full year, so that buys you a LOT of time for you to build better tasks.
3. Clearly AI agents need much higher success rates than 80%. Investigate 95% success rate measurements in the future for further savings as your tasks begin to become saturated.



LIKE (4)



REPLY

23 more comments...

Cookie Policy

We use cookies to improve your experience, for analytics, and for marketing. You can accept, reject, or manage your preferences. See our [privacy policy](#).

Manage **Reject** **Accept**